Firebase: O Guia Completo

O que é o Firebase?

Imagine que você está construindo um aplicativo. Além das telas e funcionalidades que o usuário vê, você precisa de uma estrutura de *backend*: um lugar para salvar dados, gerenciar logins de usuários, enviar notificações e muito mais. Construir tudo isso do zero é complexo, demorado e exige manutenção constante.

É aqui que o Firebase entra. O Firebase é uma plataforma do Google que funciona como um **Backend-como-um-Serviço (BaaS)**. Pense nela como uma caixa de ferramentas completa e pronta para uso, que cuida de toda a infraestrutura complexa do backend. Ao usar o Firebase, desenvolvedores podem focar no que realmente importa: criar uma experiência incrível para o usuário, sem se preocupar com a configuração, escalabilidade ou segurança dos servidores.

Por que escolher o Firebase?

Se você ainda está se perguntando sobre os benefícios práticos, aqui estão as principais razões para adotar o Firebase em seus projetos:

- **Desenvolvimento Acelerado:** Com funcionalidades prontas como autenticação e banco de dados, você economiza semanas ou até meses de trabalho que seriam gastos construindo essa base do zero.
- **Facilidade de Integração:** O Firebase oferece SDKs (Kits de Desenvolvimento de Software) otimizados para diversas linguagens e plataformas, incluindo **Flutter, JavaScript, Java, Swift, Node.js** e outras. A documentação é clara e repleta de exemplos, facilitando a integração.
- **Escalabilidade Automática:** Construído sobre a infraestrutura do Google Cloud, seu aplicativo pode começar pequeno e crescer para milhões de usuários sem que você precise gerenciar um único servidor. O Firebase escala automaticamente conforme a demanda.
- **Ecossistema Completo:** Oferece um vasto leque de serviços integrados, desde bancos de dados em tempo real e autenticação até machine learning, armazenamento de arquivos e hospedagem.

- **Análise e Engajamento:** Ferramentas como o **Firebase Analytics** permitem entender o comportamento dos seus usuários, enquanto o **Crashlytics** fornece relatórios detalhados de falhas, ajudando a melhorar a estabilidade e a qualidade do seu app.
- **Integração com o Google:** Por ser um produto Google, ele se integra perfeitamente com outras ferramentas poderosas como Google Ads, Google Play e Google Marketing Platform.

Principais Funcionalidades do Firebase

O Firebase é um conjunto de ferramentas. Abaixo estão as mais utilizadas:

- **Firebase Authentication:**
    - **Descrição:** Um sistema de autenticação completo e seguro. Permite que os usuários façam login com e-mail/senha, números de telefone ou provedores populares como Google, Facebook, Twitter e GitHub.
    - **Uso:** Gerenciar a identidade dos usuários e proteger os dados do seu aplicativo.

- **Cloud Firestore:**
    - **Descrição:** Um banco de dados NoSQL, flexível e escalável, para armazenar e sincronizar dados em tempo real entre os clientes.
    - **Uso:** Ideal para a maioria das aplicações que precisam de armazenamento dinâmico de dados, como perfis de usuário, posts, produtos, etc.

- **Realtime Database:**
    - **Descrição:** O banco de dados NoSQL original do Firebase. Sincroniza dados em milissegundos, sendo extremamente rápido para operações em tempo real.
    - **Uso:** Perfeito para aplicativos que exigem sincronização instantânea, como chats ao vivo ou documentos colaborativos.

- **Cloud Functions:**
    - **Descrição:** Permite executar código de backend (JavaScript/TypeScript) em resposta a eventos, como a criação de um novo usuário ou a gravação de um dado no banco, sem a necessidade de gerenciar um servidor.
    - **Uso:** Automatizar tarefas, como enviar um e-mail de boas-vindas, processar imagens ou interagir com outras APIs.

- **Firebase Cloud Messaging (FCM):**
    - **Descrição:** Um serviço para enviar notificações push e mensagens para seus usuários de forma gratuita e confiável.
    - **Uso:** Manter os usuários engajados, enviando alertas, novidades e outras informações importantes.

- **Firebase Hosting:**
    - **Descrição:** Um serviço de hospedagem rápido, seguro e com CDN global para conteúdo web (sites, landing pages, PWAs).
    - **Uso:** Publicar seu site ou aplicativo web com um único comando e garantir que ele seja entregue rapidamente em qualquer lugar do mundo.

Projeto Prático: Criando um App de Lista de Tarefas com Flutter e Firebase

Vamos colocar a mão na massa! Este passo a passo guiará você na criação de um aplicativo de lista de tarefas (To-do List) totalmente funcional, com autenticação de usuários e armazenamento de dados em tempo real.

Fase 1: Configuração do Ambiente Firebase

Antes de escrever qualquer código, precisamos preparar nosso projeto no Firebase.

1.1. Crie um Projeto no Firebase Console:

1. Acesse o Firebase Console.
2. Clique em "**Adicionar projeto**".

3. Dê um nome ao seu projeto (ex: lista-de-tarefas-flutter) e siga as instruções na tela. Pode desativar o Google Analytics por agora para simplificar.

1.2. Ative os Serviços Necessários:

No menu lateral do seu novo projeto, vamos ativar os serviços que usaremos:

1. **Authentication:**
    - Vá para **Build > Authentication**.
    - Clique em "**Primeiros passos**".
    - Na aba "**Sign-in method**", selecione "**E-mail/senha**" e ative-o.

2. **Firestore:**
    - Vá para **Build > Firestore Database**.
    - Clique em "**Criar banco de dados**".
    - Inicie no **modo de teste** (ele permite leituras e escritas sem regras de segurança por 30 dias, ideal para desenvolvimento).
    - Escolha um local para o servidor (geralmente southamerica-east1 para o Brasil).

1.3. Instale e Configure a Firebase CLI:

A Firebase CLI (Interface de Linha de Comando) nos ajudará a conectar nosso app Flutter ao projeto Firebase.

1. **Instale o Node.js:** Se ainda não o tiver, baixe e instale a partir do site oficial.
2. **Instale a Firebase CLI:** Abra seu terminal (Prompt de Comando, PowerShell ou Terminal) e execute o comando:

    npm install -g firebase-tools
3. **Faça login na sua conta Google:**

    firebase login
4. Isso abrirá uma janela no seu navegador para você autorizar o acesso.
5. **Verifique os seus PRojetos do FireBase**

    firebase projects:list (lista os projetos)

Fase 2: Configuração do Projeto Flutter

Agora, vamos criar nosso app Flutter e conectá-lo ao Firebase.

2.1. Crie um Novo Projeto Flutter:

Flutter Create

2.2. Adicione as Dependências:

firebase_core

firebase_auth

cloud_firestore

flutter pub get

2.3. Conecte o Flutter ao Firebase (A Mágica Acontece Aqui):

Esta é a ferramenta auxiliar que conecta especificamente projetos Flutter.

1. **Execute o Comando de Ativação:** Esta ferramenta é um pacote da linguagem Dart, então usamos o gerenciador de pacotes do Dart (pub). No terminal, execute:

   dart pub global activate flutterfire_cli

2. **Atenção ao AVISO DE PATH (Problema Comum):** Após a instalação, você provavelmente verá uma mensagem de aviso (Warning) parecida com esta:

   Warning: Pub installs executables into C:\Users\SeuUsuario\AppData\Local\Pub\Cache\bin, which is not on your

path.You can fix that by adding that directory to your system's "Path" environment variable.

3. **Isso é extremamente importante!** O Dart instalou o comando flutterfire, mas seu terminal não sabe onde encontrá-lo. Você precisa modificar o caminho do arquivo (C:\Users\SeuUsuario\AppData\Local\Pub\Cache\bin no exemplo) para C:/src/flutter/bin

2.4. Conectar seu Projeto Flutter ao Firebase (A Etapa Final)

Agora que tudo está instalado e configurado, o processo de conectar seu app é muito simples.

1. **Navegue até a Pasta do seu Projeto:** Pelo terminal, entre na pasta raiz do seu projeto Flutter.
2. **Execute o Comando de Configuração:**

```
flutterfire configure
```

3. **Processo Interativo:**
    - A CLI irá se conectar à sua conta Firebase (que você já autenticou no Passo 2).
    - Ela exibirá a lista dos seus projetos Firebase. Use as setas do teclado para selecionar o projeto desejado e pressione Enter.
    - Em seguida, ela perguntará para quais plataformas você deseja configurar o projeto (Android, iOS, Web, etc.). Selecione as desejadas com a barra de espaço e pressione Enter.
    - **A mágica acontece:** A ferramenta irá se comunicar com o Firebase, registrar seus apps (criando apps para Android/iOS/Web dentro do projeto Firebase) e, o mais importante, **gerará o arquivo** no seu projeto Flutter.

Resumo Rápido dos Comandos

Comando	Descrição	Quando Usar
npm install -g firebase-tools	Instala a CLI principal do Firebase.	Apenas uma vez para configurar a máquina.
firebase login	Autentica sua máquina com sua conta Google.	Apenas uma vez, ou se precisar trocar de conta.
firebase projects:list	Lista os projetos Firebase da sua conta.	Para verificar se o login funcionou.
dart pub global activate flutterfire_cli	Instala a CLI específica para Flutter.	Apenas uma vez para configurar a máquina.
flutterfire configure	Conecta seu projeto Flutter a um projeto Firebase.	Uma vez por cada novo projeto Flutter que você criar.

Fase 3: Construindo as Telas da Aplicação

https://docs.google.com/document/d/115PAyV8CaJt5vHO2sZkhjHS8k7WtpJaCBtE6ldzj2n0/edit?tab=t.0

Atividade Formativa - Criando o App "CineFavorite" com Flutter e Firebase

Construindo um aplicativo completo do zero. Nosso app, o CineFavorite, permitirá que se crie uma conta, busque filmes e monte uma galeria pessoal de favoritos, com capas e notas.

Tecnologias que vamos usar:

- Flutter: Para criar uma interface bonita e funcional para Android e iOS.
- Firebase: Nosso backend na nuvem, que cuidará de:
    - Authentication: Login e registro de usuários.
    - Cloud Firestore: Nosso banco de dados para salvar a lista de filmes.
- API do TMDb: Um serviço externo que nos fornecerá os dados e as imagens dos filmes.

Fase 0: Construção dos Protótipo do Projeto

- Tela de Login
- Tela de Registro
- Tela de Procura de Filmes
- Tela de Favoritos (Home)

Fase 1: Preparando a Fundação (Configuração do Firebase e API)

Antes de escrever qualquer código, precisamos preparar nosso servidor na nuvem.

Passo 1.1: Criando o Projeto no Firebase

1. Abra o navegador e acesse o Firebase Console.
2. Clique em "Adicionar projeto".
3. Dê um nome ao projeto, como cinefavorite-turma-app, e siga os passos de criação. Pode desmarcar a opção do Google Analytics para simplificar.

Passo 1.2: Ativando os Serviços do Firebase

No menu lateral esquerdo do seu projeto, vamos ativar os "tijolos" que precisamos:

1. Authentication (Autenticação):

    - Vá em Build > Authentication.
    - Clique em "Primeiros Passos".
    - Na aba "Métodos de login", selecione "E-mail/senha" e ative-o.
2. Firestore Database (Banco de Dados):
    - Vá em Build > Firestore Database.
    - Clique em "Criar banco de dados".
    - Inicie no modo de teste. Isso nos permite ler e escrever dados facilmente durante o desenvolvimento.
3. Storage (Armazenamento de Arquivos):
    - Vá em Build > Storage.
    - Clique em "Primeiros passos" e configure-o também no modo de teste.

Passo 1.3: Obtendo a Chave da API de Filmes (TMDb)

Nosso app precisa de um lugar para buscar os filmes. Usaremos o The Movie Database (TMDb).

1. Acesse The Movie Database (TMDb) e crie uma conta gratuita.
2. Clique no seu ícone de perfil, vá em Configurações > API.
3. Siga as instruções para solicitar uma chave de API (API Key).
4. Copie essa chave e guarde-a em um bloco de notas. Vamos precisar dela no nosso código!

Fase 2: Configurando o Projeto Flutter

Agora, vamos criar nosso projeto local e conectá-lo ao Firebase.

Passo 2.1: Criando o Projeto Flutter

No seu terminal, execute:

flutter create cine_favorite
cd cine_favorite

adicione as seguintes dependências

```
# Pacotes do Firebase
firebase_core
firebase_auth
```

```
cloud_firestore
firebase_storage

# Pacote para fazer chamadas à API
http

# Pacotes para melhorar a UI
cached_network_image
flutter_rating_bar
```

Passo 2.3: Conectando o App ao Firebase

No terminal, na raiz do projeto, execute o comando mágico da FlutterFire CLI:

```
flutterfire configure
```

Siga as instruções: selecione sua conta, escolha o projeto cinefavorite-turma-app que criamos e confirme as plataformas (Android). Este comando criará o arquivo lib/firebase_options.dart que faz a conexão.

Fase 3: Escrevendo o Código (A Arquitetura)

Vamos organizar nosso código de forma limpa, separando as responsabilidades. Crie as seguintes pastas dentro de lib: models, controllers, e screens.

Passo 3.1: O Modelo de Dados (A "Planta" do nosso Filme)

Arquivo: lib/models/movie.dart
Este arquivo define como é um objeto "Filme" no nosso app.

Passo 3.2: Os Controllers (O "Cérebro" do App)

Aqui fica toda a lógica de comunicação com a internet (Firebase e TMDb).

Arquivo: lib/controllers/tmdb_controller.dart
Este controller é responsável APENAS por falar com a API do TMDb.

Arquivo: lib/controllers/firebase_controller.dart
Este é o nosso controller mais importante. Ele gerencia tudo relacionado ao Firebase.

Passo 3.3: As Telas (A Parte Visual)

Arquivo: lib/main.dart (Ponto de Partida e Roteador)

Este arquivo decide se mostra a tela de Login ou a tela Principal.

Arquivo: lib/screens/login_screen.dart e lib/screens/register_screen.dart
Estas telas são simples: apenas campos de texto e botões para autenticação.

Arquivo: lib/screens/search_screen.dart (A Busca Inteligente)

Arquivo: lib/screens/home_screen.dart (A Galeria Principal)

Situação de Aprendizagem Somativa

Chegamos ao momento de consolidar e celebrar todo o conhecimento que construímos juntos ao longo deste curso. Esta atividade somativa final não é apenas um teste, mas uma oportunidade para vocês atuarem como profissionais do mercado, transformando uma ideia em um produto real, funcional e de alta qualidade.

Vocês terão a chance de escolher um entre cinco cenários de projeto, cada um com um desafio único, que simula as demandas de uma startup de tecnologia. Seja construindo um caderno de receitas, uma estante de livros ou um diário de viagens, o objetivo é o mesmo: aplicar suas habilidades em Flutter e Firebase para criar uma solução robusta, criativa e centrada no usuário.

Este é o seu projeto de portfólio. É a sua chance de demonstrar não apenas que sabem programar, mas que conseguem resolver problemas, planejar, executar e entregar um software do qual se orgulham.

Objetivo Geral da Atividade

O objetivo principal desta avaliação é aferir sua capacidade de projetar, desenvolver e implantar uma aplicação móvel full-stack utilizando o ecossistema Flutter e Firebase. Você deverá demonstrar proficiência desde a concepção da arquitetura de dados no backend até a construção de uma interface de usuário funcional e intuitiva no frontend, validando as competências técnicas e socioemocionais essenciais para um desenvolvedor de software moderno.

Escopo Geral do Projeto

Independentemente do tema escolhido, todos os projetos deverão contemplar o seguinte escopo mínimo:

1. **Backend na Nuvem:** Configuração e utilização de um projeto Firebase, incluindo:
    - **Autenticação de Usuários:** Sistema seguro de login e registro.
    - **Banco de Dados:** Modelagem e manipulação de dados com o Cloud Firestore..
2. **Frontend Responsivo:** Desenvolvimento de uma aplicação em Flutter com:
    - Navegação clara entre múltiplas telas.
    - Interface que consome dados de uma API externa (TMDb, TheMealDB, Google Books, etc.).
    - Interação em tempo real com o backend do Firebase.

3. **Entrega Final:** O projeto deve ser entregue por meio de um repositório no GitHub, contendo todo o código-fonte funcional e um arquivo `README.md` com as instruções para executar a aplicação.

Critérios de Avaliação

Sua avaliação será baseada em duas frentes complementares: suas habilidades técnicas (o que você fez) e suas habilidades socioemocionais (como você fez).

Critérios de Avaliação Técnicos

- **Proficiência em Firebase e Backend:**
    - **Modelagem de Dados:** Estrutura de dados no Firestore é lógica, eficiente e escalável.
    - **Regras de Segurança:** As regras do Firestore e Storage são robustas e garantem a privacidade e a integridade dos dados do usuário.
    - **Integração de Serviços:** Utilização correta e combinada dos serviços do Firebase (Auth, Firestore, Storage, Cloud Functions, etc.) conforme o escopo do projeto escolhido.
- **Qualidade do Código Flutter e Arquitetura:**
    - **Arquitetura Limpa:** O código é bem organizado, seguindo padrões de separação de responsabilidades (ex: UI, lógica de negócio, serviços).
    - **Qualidade e Legibilidade:** O código é limpo, bem documentado, segue as convenções da linguagem Dart (`dart format`) e utiliza nomes significativos para variáveis e funções.
    - **Gerenciamento de Estado:** O estado da aplicação é gerenciado de forma eficiente e previsível.
- **Funcionalidade e Experiência do Usuário (UX):**
    - **Conformidade com o Escopo:** Todas as funcionalidades descritas no cenário escolhido foram implementadas e funcionam como esperado.
    - **Robustez e Tratamento de Erros:** O aplicativo lida com cenários de erro (ex: falta de conexão, falhas na API) de forma elegante, fornecendo feedback claro ao usuário.
    - **Intuitividade:** A navegação e o uso da aplicação são claros e fáceis de entender.

Critérios de Avaliação Socioemocionais

- **Resolução de Problemas e Autonomia:**

    - Demonstra capacidade de pesquisar soluções para desafios técnicos de forma independente.
    - Aplica a lógica para depurar e corrigir bugs de forma eficaz, sem depender excessivamente de ajuda externa.
    - Mostra criatividade ao encontrar soluções para os requisitos do projeto.
- **Planejamento e Organização:**
    - **Gestão de Projeto (Git/GitHub):** Utiliza o Git para controle de versão de forma consistente, com commits claros e significativos.
    - **Documentação:** O arquivo `README.md` é bem escrito, explicando o propósito do projeto e como executá-lo.
    - **Gerenciamento do Tempo:** O projeto é entregue dentro do prazo estipulado, com o escopo proposto completo.
- **Mentalidade de Crescimento e Qualidade:**
    - **Atenção aos Detalhes:** O produto final é polido, sem erros óbvios de UI ou bugs críticos.
    - **Proatividade:** Vai além do mínimo exigido, adicionando pequenos refinamentos que melhoram a experiência do usuário (feedback visual, microanimações, etc.).
    - **Receptividade a Feedback (se aplicável):** Demonstra capacidade de incorporar sugestões e melhorar o projeto com base em orientações.

Situação de Aprendizagem 1: O Caderno de Receitas Digital ("Chef's Shelf")

Foco da Avaliação: Modelagem de dados complexos no Firestore (arrays e maps) e interação CRUD (Criar, Ler, Atualizar, Deletar).

- **Briefing:**
    - **Cliente:** Startup "GourmetGo".
    - **Missão:** Estamos criando um aplicativo onde os usuários podem descobrir receitas online e salvá-las em seu caderno de receitas pessoal. O desafio é que uma receita não é apenas um título; ela tem uma lista de ingredientes e uma sequência de passos. Sua tarefa é construir o núcleo desta aplicação, garantindo que os dados das receitas sejam armazenados de forma estruturada e eficiente no Firebase.
- **Escopo do Projeto:**
    - **API:** Integrar com uma API de receitas, como a **TheMealDB** (gratuita e de fácil uso).
    - **Autenticação:** Sistema de login/registro para que cada usuário tenha seu próprio caderno.

    - **Busca e Visualização:** Uma tela para buscar receitas da API e visualizar os detalhes (imagem, ingredientes, instruções).
    - **Funcionalidade Principal ("Salvar no Caderno"):**
        - Ao salvar uma receita, os dados devem ser copiados da API e armazenados no Firestore do usuário.
        - A estrutura no Firestore deve ser bem organizada, salvando a lista de ingredientes como um `array` de `strings` e as instruções como um `map` ou `array` de `strings`.
    - **Tela "Meu Caderno":** Exibir a lista de receitas salvas pelo usuário, lendo os dados do Firestore. Ao clicar em uma receita, o usuário deve ver os detalhes completos.
    - **Deleção:** Permitir que o usuário remova uma receita de seu caderno.
- **Critérios de Avaliação Especifico:**
    - **Modelagem de Dados no Firestore:** A estrutura do documento da receita é eficiente e usa corretamente tipos de dados complexos (arrays/maps) para ingredientes e instruções.
    - **Implementação do CRUD:** As funcionalidades de salvar (Create), ler (Read) e deletar (Delete) receitas no caderno do usuário funcionam perfeitamente.
    - **Integração com API:** A busca e a exibição de dados da API TheMealDB são funcionais.
    - **Autenticação e Regras de Segurança:** O sistema de login funciona e as regras do Firestore garantem que um usuário só possa acessar seu próprio caderno de receitas.

Situação de Aprendizagem 2: A Estante de Livros Digital ("Book Nook")

Foco da Avaliação: Lógica de backend com Cloud Functions e gerenciamento de estado dos dados.

- **Briefing:**
    - **Cliente:** Clube do Livro "Leitores & Cia.".
    - **Missão:** Precisamos de um aplicativo para que nossos membros possam organizar suas leituras. Eles devem poder adicionar livros a diferentes "estantes" virtuais ("Quero Ler", "Lendo", "Lidos"). O grande diferencial é que queremos um painel de estatísticas que seja atualizado automaticamente. Sua missão é construir essa funcionalidade, com foco na automação do backend.
- **Escopo do Projeto:**
    - **API:** Integrar com a **Google Books API** para buscar livros por título ou autor.

    - **Autenticação:** Sistema de login/registro para cada membro do clube.
    - **Gerenciamento de Estantes:** O usuário pode buscar um livro e adicioná-lo a uma de três estantes. O estado (`status`) do livro deve ser salvo no Firestore.
    - **Funcionalidade Principal (Cloud Function):**
        - Escrever e implantar uma Cloud Function que é acionada (`onUpdate`) sempre que o `status` de um livro de um usuário é alterado para "Lido".
        - A função deve ler os dados do livro (ex: número de páginas) e atualizar um documento de estatísticas do usuário (ex: `/users/{userId}/stats/`), incrementando contadores como `livrosLidos` e `paginasLidas`.
    - **Tela "Minhas Estatísticas":** Uma tela no app que lê e exibe os dados do documento de estatísticas do usuário, mostrando quantos livros e páginas ele já leu.
- **Critérios de Avaliação Especifico:**
    - **Implementação da Cloud Function:** A função é acionada corretamente, contém a lógica de negócio para atualizar as estatísticas e manipula os dados no Firestore de forma segura e eficiente.
    - **Gerenciamento de Estado no App:** O usuário consegue mover livros entre as estantes e a mudança é refletida corretamente no Firestore.
    - **Leitura e Exibição de Dados Agregados:** A tela de estatísticas exibe os dados gerados pela Cloud Function.
    - **Qualidade do Código Backend:** O código da Cloud Function (JavaScript/TypeScript) é limpo, legível e seguro.

Situação de Aprendizagem 3: O Log de Viagens ("WanderLog")

Foco da Avaliação: Integração de Firebase Storage para conteúdo gerado pelo usuário e uso de tipos de dados de geolocalização.

- **Briefing:**
    - **Cliente:** Agência de Viagens "Mundo Afora".
    - **Missão:** Queremos oferecer aos nossos clientes um aplicativo onde eles possam criar um diário visual de suas viagens. Eles devem poder registrar um local que visitaram, escrever uma pequena memória sobre ele e, o mais importante, anexar uma foto pessoal daquele momento. Sua tarefa é construir o sistema que lida com o upload de imagens e o armazenamento de dados geográficos.
- **Escopo do Projeto:**
    - **API (Opcional, mas recomendado):** Integrar com uma API de geolocalização como **Google Places** ou **OpenStreetMap** para buscar e autocompletar nomes de lugares.

    - **Autenticação:** Sistema de login/registro.
    - **Tela de Novo Registro:** Um formulário onde o usuário pode:
        - Digitar o nome de um local.
        - Escrever uma breve descrição ou memória.
        - Selecionar uma foto da galeria do seu celular.
        - (Bônus) Capturar as coordenadas geográficas atuais.
    - **Funcionalidade Principal (Storage e Firestore):**
        - Ao salvar o registro, a foto selecionada deve ser enviada para o **Firebase Storage** em uma pasta específica do usuário.
        - Após o upload, a URL de download da imagem, junto com o nome do local, a descrição e as coordenadas (`GeoPoint`), deve ser salva em um documento no **Firestore**.
    - **Tela Principal:** Exibir os registros de viagem do usuário em um formato visual (ex: cartões com a foto e o título).
- **Critérios de Avaliação Específicos**
    - **Integração com Firebase Storage:** O fluxo de seleção, upload da imagem e obtenção da URL de download está funcionando corretamente e de forma segura.
    - **Armazenamento de Dados Estruturados:** O documento no Firestore armazena corretamente a URL do Storage e outros tipos de dados, como `GeoPoint`.
    - **Experiência do Usuário no Upload:** O app fornece feedback ao usuário durante o upload da imagem (ex: `CircularProgressIndicator`).
    - **Regras de Segurança:** As regras do Storage e do Firestore garantem que um usuário só possa fazer upload e visualizar seu próprio conteúdo.

Situação de Aprendizagem 4: A Discoteca Pessoal ("Discotheque")

Foco da Avaliação: Relações de dados (muitos-para-muitos) e segurança em dados compartilhados.

- **Briefing:**
    - **Cliente:** App de Música "GrooveShare".
    - **Missão:** O recurso mais pedido pelos nossos usuários é a criação de playlists colaborativas. Precisamos de um sistema onde um usuário possa criar uma playlist, adicionar músicas e convidar amigos para adicionar músicas também. O desafio está na modelagem de dados e nas regras de segurança para permitir que múltiplos usuários editem a mesma lista.
- **Escopo do Projeto:**

    - **API:** Integrar com a API do **Spotify** ou **Last.fm** para buscar álbuns e faixas.
    - **Autenticação:** Sistema de login/registro.
    - **Modelagem de Dados:** Projetar uma estrutura no Firestore para representar a relação muitos-para-muitos: usuários podem ter múltiplas playlists, e playlists podem ter múltiplos colaboradores.
        - Sugestão: Uma coleção de `playlists` onde cada documento tem um campo `ownerId` e um `array` `collaborators` com os UIDs dos membros.
    - **Funcionalidade Principal (Colaboração):**
        - Um usuário pode criar uma playlist.
        - O dono da playlist pode convidar outro usuário (ex: digitando seu e-mail) para se tornar um colaborador.
        - Tanto o dono quanto os colaboradores podem adicionar ou remover músicas da playlist.
    - **Regras de Segurança Avançadas:** As regras do Firestore devem garantir que:
        - Apenas o dono da playlist pode convidar ou remover colaboradores.
        - Apenas o dono e os colaboradores listados no `array` podem modificar as músicas da playlist.
- **Critérios de Avaliação Específicos:**
    - **Implementação das Regras de Segurança:** As regras são complexas e garantem com sucesso as permissões de acesso e escrita colaborativa, sendo o principal ponto da avaliação.
    - **Modelagem de Dados:** A estrutura de dados para playlists e colaboradores é eficiente e escalável.
    - **Funcionalidade da Aplicação:** O fluxo de criação de playlists, convite de amigos e adição/remoção de músicas funciona como esperado.

Situação de Aprendizagem 5: O Gerador de Citações ("QuoteGenius")

Foco da Avaliação: Integração com APIs de terceiros via Cloud Functions e entrega de dados em tempo real com o Firestore.

- **Briefing:**
    - **Cliente:** App de Bem-Estar "ZenMind".
    - **Missão:** Queremos ter uma seção no nosso app chamada "Citação do Dia". Não queremos depender de uma lista fixa de citações no app. Em vez disso, queremos que nosso servidor busque uma nova citação de uma API externa todos os dias à

meia-noite e a disponibilize para todos os usuários. Sua tarefa é construir esse sistema de backend automatizado.

- **Escopo do Projeto:**
    - **API:** Utilizar uma API de citações, como a **Quotable** ou **FavQs**.
    - **Cloud Function Agendada (Scheduled):**
        - Criar uma Cloud Function que é configurada para rodar automaticamente em um horário fixo (ex: todos os dias às 00:01) usando o agendador do Firebase.
        - Dentro da função, fazer uma chamada à API de citações para buscar uma citação aleatória.
        - Salvar a citação obtida (texto e autor) em um documento específico e bem conhecido no Firestore (ex: `/dailyContent/quoteOfTheDay`).
    - **App Flutter:**
        - A tela principal do app não precisa de autenticação.
        - Ela deve usar um `StreamBuilder` para "ouvir" as mudanças no documento `/dailyContent/quoteOfTheDay`.
        - Quando a Cloud Function atualizar o documento à meia-noite, a citação na tela de todos os usuários deve mudar automaticamente, em tempo real.
- **Critérios de Avaliação Específicos:**
    - **Implementação da Cloud Function Agendada:** A função está corretamente configurada para rodar no horário, chama a API externa e atualiza o Firestore com sucesso.
    - **Sincronização em Tempo Real no App:** O aplicativo Flutter utiliza `StreamBuilder` para ouvir o documento e exibe a citação, atualizando-a em tempo real quando os dados mudam no backend.
    - **Interação Segura entre Serviços:** A Cloud Function interage com o Firestore usando permissões de administrador de forma segura.
    - **Robustez:** A função lida com possíveis erros da chamada de API (ex: usando `try-catch`).